

数据读取

数据读取

- 1. 快速上手
- 2. 硬件接线
- 3. 使用方法
- 4. 现象结果
- 5. 代码解释

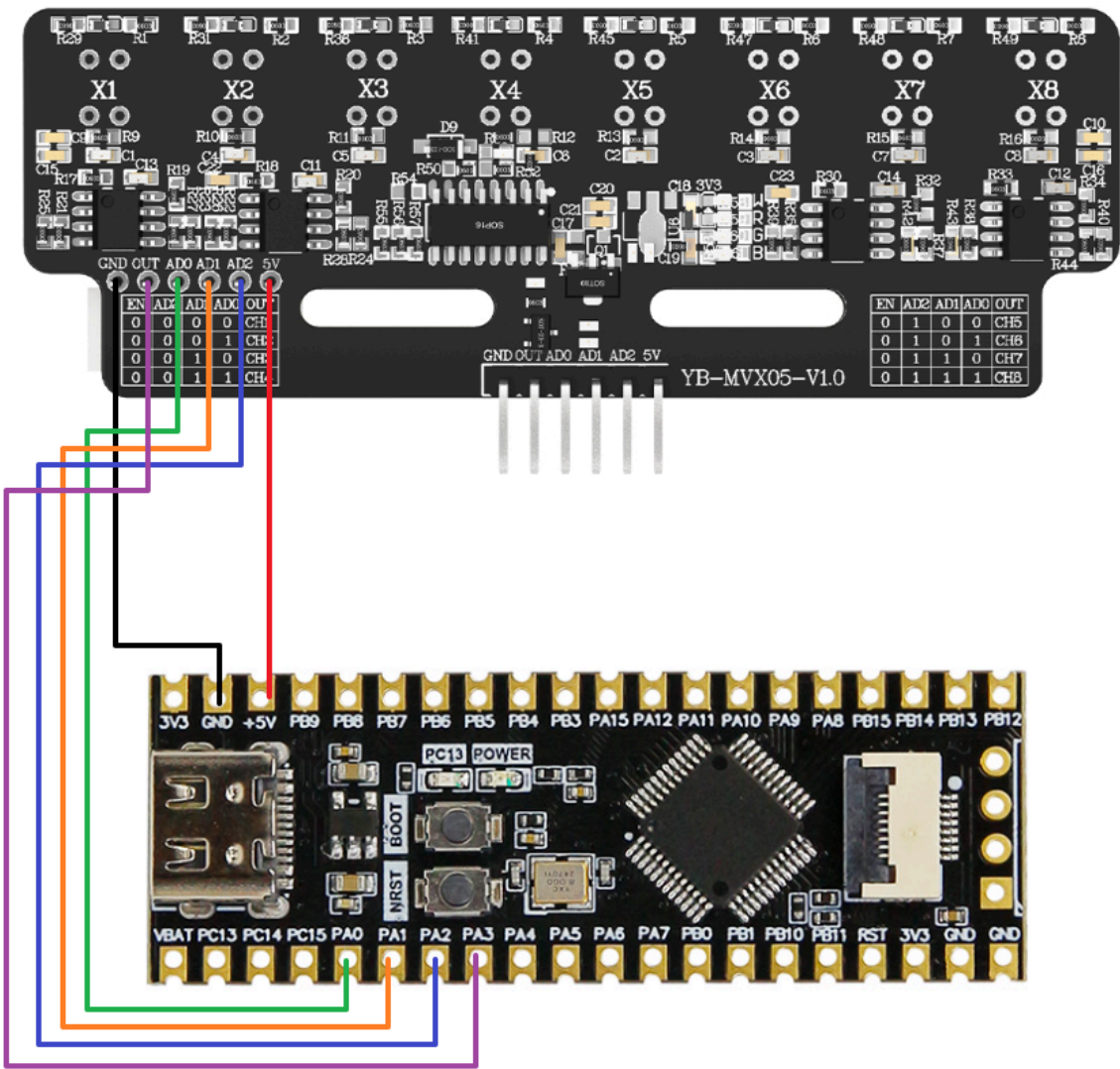
1. 快速上手

本教程讲解如何使用 STM32F103C8T6 主板读取八路灰度巡线模块的数字量信息，并通过串口助手打印结果。

教程中使用的硬件均是亚博售卖的 STM32F103C8T6 核心板、八路灰度巡线模块。查看下方的接线图进行接线后，再给主板烧录程序，就可以获得数据了。非亚博的模块仅供参考。

2. 硬件接线

STM32主板的typec口需要与电脑的USB口相连



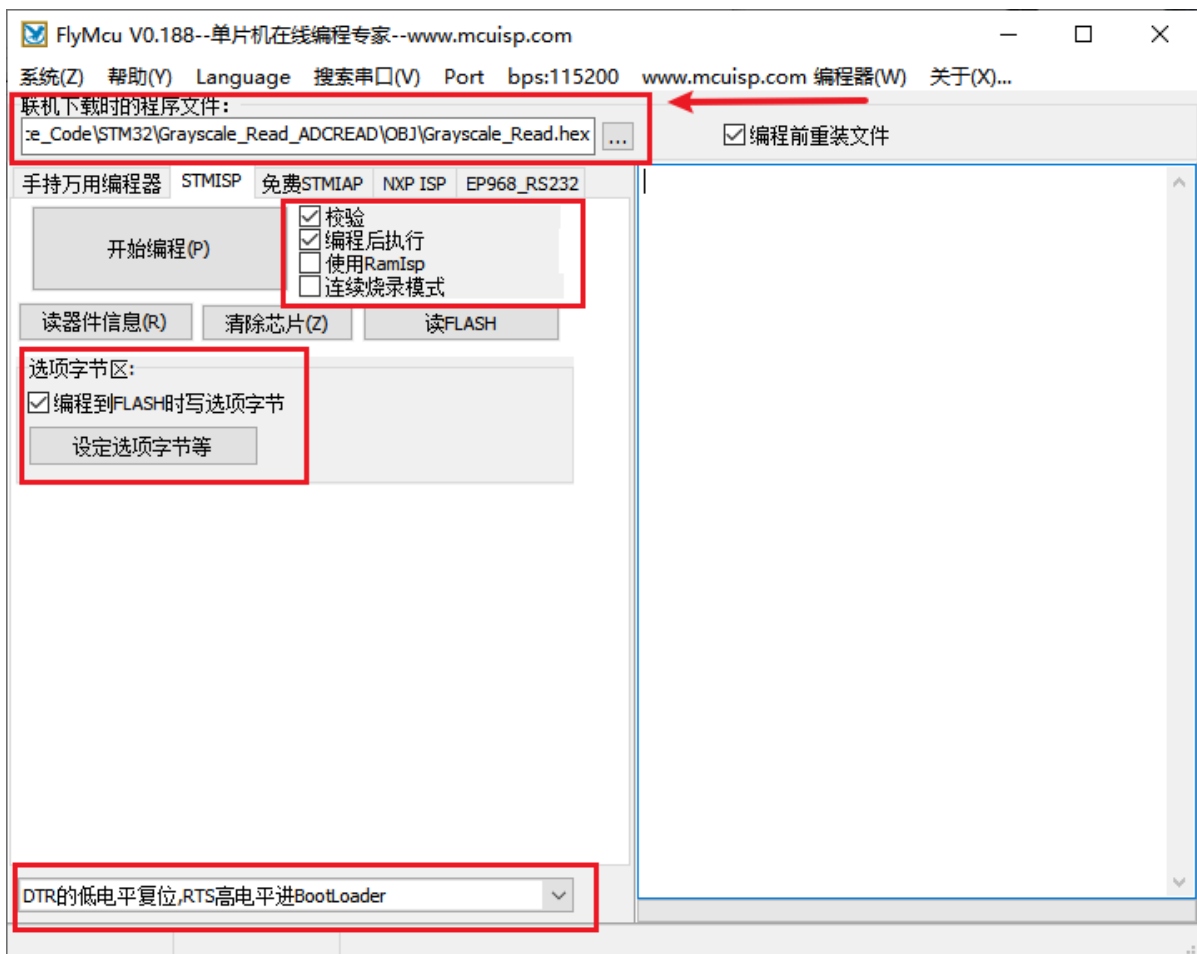
八路灰度巡线模块	STM32F103C8T6
5V	5V
GND	GND
AD0	PA0
AD1	PA1
AD2	PA2
OUT	PA3

亚博售卖的八路灰度模块连接使用的线材为XH2.54转6pin杜邦线：

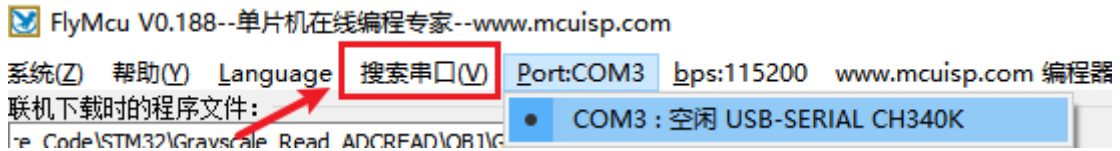


3. 使用方法

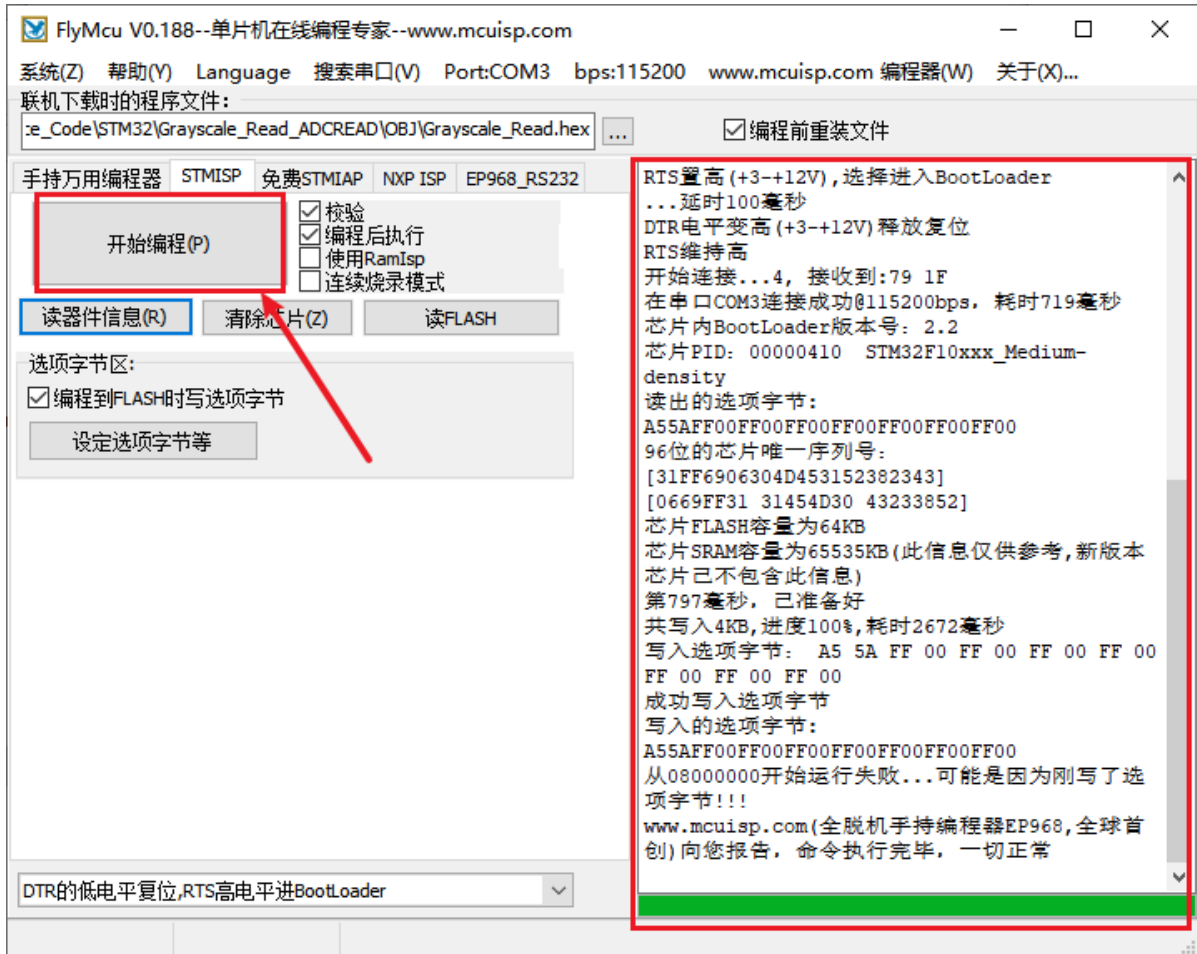
1. 打开FlyMcu烧录工具，程序文件选择代码工程里面的hex文件，其他红框框起来的地方需要与图片保持一致。



2. 将STM32与电脑连接，点击搜索串口，然后选择STM32属于的COM口。



3. 然后点击开始编程，程序开始被烧入到主板中，等待右边的进度条走完，也可以看到烧录成功的信息



4. 按一下主板上的NRST复位键，重启主板运行程序，然后打开串口助手查看打印的信息。

4. 现象结果

在串口助手中，您可以看到持续打印的八路灰度巡线模块的数字量。X1 对应模块上的 X1，当 X1 的灯亮起时，对应的值就为 1。

```
[2025-11-08 17:18:48.215]# RECV ASCII>
[ X1:1 ][ X2:1 ][ X3:1 ][ X4:1 ][ X5:1 ][ X6:1 ][ X7:1 ][ X8:1 ]

[2025-11-08 17:18:48.312]# RECV ASCII>
[ X1:1 ][ X2:1 ][ X3:1 ][ X4:1 ][ X5:1 ][ X6:1 ][ X7:1 ][ X8:1 ]

[2025-11-08 17:18:48.407]# RECV ASCII>
[ X1:1 ][ X2:1 ][ X3:1 ][ X4:1 ][ X5:1 ][ X6:1 ][ X7:1 ][ X8:1 ]

[2025-11-08 17:18:48.502]# RECV ASCII>
[ X1:1 ][ X2:1 ][ X3:1 ][ X4:1 ][ X5:1 ][ X6:1 ][ X7:1 ][ X8:1 ]

[2025-11-08 17:18:48.613]# RECV ASCII>
[ X1:1 ][ X2:1 ][ X3:1 ][ X4:1 ][ X5:1 ][ X6:1 ][ X7:1 ][ X8:1 ]

[2025-11-08 17:18:48.709]# RECV ASCII>
[ X1:1 ][ X2:1 ][ X3:1 ][ X4:1 ][ X5:1 ][ X6:1 ][ X7:1 ][ X8:1 ]
```

5. 代码解释

```
// main.c
int main(void)
{
    bsp_init(); // 初始化板载外设，包括串口和延时定时器
    Grayscale_Sensor_Init(); // 初始化灰度传感器GPIO

    while(1)
    {
        Grayscale_Sensor_Read_All(g_sensor_data);

        for (i = 0; i < GRAYSCALE_SENSOR_CHANNELS; i++)
        {
            printf("[ %s:%d ]", sensor_labels[i], g_sensor_data[i]);
        }
        printf("\n");
        delay_ms(60);
    }
}
```

- `main`: 程序主入口。首先调用 `bsp_init()` 初始化所有板级支持包 (BSP) 硬件，这其中包含了串口和Systick延时。接着调用 `Grayscale_Sensor_Init()` 初始化传感器IO口。在一个无限循环中，程序不断调用 `Grayscale_Sensor_Read_All()` 获取传感器数据，并通过 `printf` 打印出来。

```
// grayscale_sensor.c
void Grayscale_Sensor_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStructure;
```

```

RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);

GPIO_InitStructure.GPIO_Pin = GPIO_Pin_0|GPIO_Pin_1|GPIO_Pin_2;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
GPIO_Init(GPIOA, &GPIO_InitStructure);

GPIO_InitStructure.GPIO_Pin = GPIO_Pin_3;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN_FLOATING;
GPIO_Init(GPIOA, &GPIO_InitStructure);
}

void Grayscale_Sensor_Read_All(uint16_t* sensor_values)
{
    uint8_t i;
    for (i = 0; i < GRAYSCALE_SENSOR_CHANNELS; i++)
    {
        _select_channel(i);
        delay_us(50);
        sensor_values[i] = Read_OUT_value();
    }
}

static void _select_channel(uint8_t channel)
{
    SENSOR_AD0_WRITE((channel >> 0) & 0x01); // bit0 -> AD0
    SENSOR_AD1_WRITE((channel >> 1) & 0x01); // bit1 -> AD1
    SENSOR_AD2_WRITE((channel >> 2) & 0x01); // bit2 -> AD2
}

```

- `Grayscale_Sensor_Init`: 使用 STM32 标准外设库函数初始化 GPIO。首先使能 `GPIOA` 的时钟，然后将 `PA0-PA2` 配置为推挽输出模式用于通道选择，将 `PA3` 配置为浮空输入模式用于数据读取。
- `Grayscale_Sensor_Read_All`: 遍历所有8个传感器通道，通过 `_select_channel` 选择通道，然后读取其值并存入数组。
- `_select_channel`: 通过 `SENSOR_ADx_WRITE` 宏来控制 GPIO 电平，从而选中对应的传感器。